CMSC 498Y Statistical Inference and Machine Learning for Genomics Data

Assignment # 6 – Mini Project

Last updated: 2026-04-19 22:56:28 -04:00

You may

- You are encouraged to work with up to 3 other students and divide and conquer the work (max 4 students per project team).
- You are encouraged to implement your approach in convenient tools like PyTorch
- You are encouraged to use AI tools (e.g., Claude or ChatGPT) to speed up all of the work for the project.

The goal of this assignment is to fine tune a protein language model for predicting protein secondary structure and then write a report on your results.

1 Approach

Your method should have the following input/output:

- **Input:** a protein sequence $x = x_1 x_2 \dots x_L$
- **Output:** the secondary structure $z = z_1 z_2 \dots z_L$, where each amino acid in the input is assigned one of three classes:
  - $H = \alpha$-helix
  - $E = \beta$-strand
  - $C =$ coil

Your approach should leverage

- A protein language model (e.g., ESM) to create a vector embedding for each amino acid in the input x (try to use the largest model that you can reasonably given your compute).
- Labeled training data created by downloading at least 100 experimentally predicted PDB structures and then calling their secondary structure with DSSP.

At the most basic level, you could have a fully connected neural network with just a few layers, but you could also be more creative in your approach.

2 Deliverables

You need to hand in a zipped folder

- PDF of report (see sections below)
- your data (names of entries from PDB, sequences, and secondary structures)
- your code (e.g., Juptyr notebooks, etc, it does not need to be well-organized).

If you don't include your data and your code, we can't give you credit for the project. Note that you can do one submission on ELMs if you let us know your project team in advance.

Your PDF report should include the following sections/information.

- (10 pts) Section about your data used for training and testing
  - List of the sequences did you downloaded from PDB
  - Present some results (i.e., plots, tables, etc.) about how similar or dissimilar the sequences are from each other and describe the trends.
    - You could compute the average vector embedding across all amino acids in the sequence and then plot it's top principle components
    - Or you could compute the L2 distance between all pairs of those vectors and then visualize the matrix and/or apply a clustering algorithm to it
    - Or you could do something else...
  - Present some results (i.e., plots, tables, etc.) related to the number of amnio acids in each class (H, E, C) and describe the trends
- (5 pts) Section about your fine-tuned model
  - Protein language model used
  - Neural network architecture (i.e., graph topology)
  - Activation functions
  - Number of learned parameters
- (5 pts) Section about your training procedure
  - Loss function and any parameter settings
  - Optimization method
  - Batch size
  - Number of epochs
  - Plot of loss function value at each epoch (or every fixed number of epochs)
- (10 pts) Section about your results of 5-fold cross-validation
  - Present results (i.e., plots, tables, etc) using at least 2 different accuracy metrics and describe the trends
  - A description of the different accuracy metrics you used and why
- (5 pts) Section about your conclusions
  - Describe whether your approach was successful, any potential limitations, and what you would try to do next if you had more time

- (5 pts) Section for acknowledgements and contributions
  - Describe the individual contributions of each group member (e.g., running analyses, visualization data, etc). Note that this can be a bullet list. And it does not need to be long.
- Miscellaneous
  - Plots and tables should have captions
  - Plots should have axis labels

3 Resources

- For using ESM, the first example on the Github Repository: https://github.com/facebookresearch/esm is helpful for completing the assignment (also see the example code below).
- For clustering based on pairwise distances, we recommend using functions in **scikit learn**; e.g. you could try using https://scikit-learn.org/stable/auto_examples/cluster/plot_agglomerative_dendrogram.html#sphx-glr-auto-examples-cluster-plot-agglomerative-dendrogram-py
- For reading and writing structure (PDB) files, we recommend using functions / data structures in **Biopython**; see this tutorial: http://biopython.org/DIST/docs/tutorial/Tutorial.html
-
- Also, you can use AI tools. It can give you a LOT of information and commands, although it can make mistakes, in which case, you need to identify the mistakes and prompt the AI to fix them.

Listing 1: Example Colab Notebook

```
pip install fair-esm
import torch
import esm

# TODO: Select model from ''Available Models and Datasets''
# on https://github.com/facebookresearch/esm
model, alphabet = esm.pretrained.esm2_t30_150M_UR50D()
nlayer = 30       # TODO: Update based on selected model
nembed = 640      # TODO: Update based on selected model

batch_converter = alphabet.get_batch_converter()
model.eval()  # disables dropout for deterministic results

# TODO: Replace with your data
# and may need to process one sequence at a time because of RAM
data = [
    ("test", "MKTVRQERLKSIVRILERSKEPVSGAQLAEELSVSRQVIVQDIAYLRSLGYNIVATPRGYVLAGG")]

batch_labels, batch_strs, batch_tokens = batch_converter(data)
batch_lens = (batch_tokens != alphabet.padding_idx).sum(1)

# Extract per-residue representations (on CPU)
with torch.no_grad():
```

```
    results = model(batch_tokens, repr_layers=[nlayer], return_contacts=True)

# Check that results includes
# ['logits', 'representations', 'attentions', 'contacts']
print([k for k in results.keys()])

# If doing Q5.1, work with final vector representations
token_representations = results["representations"][nlayer]

# Check that vector sizes match sequence length (+1) and the embedding dimension
print(token_representations.shape)
print(len(data[0][1]))
print(nembed)

# Generate per-sequence representations via averaging
# NOTE: token 0 is always a beginning-of-sequence token, so the first residue is token 1.
sequence_representations = []
for i, tokens_len in enumerate(batch_lens):
    sequence_representations.append(token_representations[i, 1 : tokens_len - 1].mean(0))
```